

π -Play: Multi-Agent Self-Play via Privileged Self-Distillation without External Data

Yaocheng Zhang^{* 1,2}, Yuanheng Zhu^{† 1,4}, Wenyue Chong^{1,2}, Songjun Tu^{1,4},
Qichao Zhang^{1,4}, Jiajun Chai³, Xiaohan Wang³, Wei Lin³,
Guojun Yin³, Dongbin Zhao^{1,2,4}

¹Institute of Automation, Chinese Academy of Sciences

²School of Advanced Interdisciplinary Sciences, University of Chinese Academy of Sciences

³Meituan ⁴School of Artificial Intelligence, University of Chinese Academy of Sciences
{zhangyaocheng2023, yuanheng.zhu}@ia.ac.cn

Abstract

Deep search agents have emerged as a promising paradigm for addressing complex information-seeking tasks, but their training remains challenging due to sparse rewards, weak credit assignment, and limited labeled data. Self-play offers a scalable route to reduce data dependence, but conventional self-play optimizes students only through sparse outcome rewards, leading to low learning efficiency. In this work, we observe that self-play naturally produces a question construction path (QCP) during task generation, an intermediate artifact that captures the reverse solution process. This reveals a new source of privileged information: self-play can provide high-quality privileged information for the self-distillation at low cost and at scale, without relying on human feedback or curated privileged information. Leveraging this insight, we propose **Privileged Information Self-Play** (π -Play), a novel multi-agent self-evolution framework combining self-play and self-distillation. In π -Play, an *examiner* generates tasks together with QCPs, and a *teacher* employs QCP as privileged context to densely supervise a *student* via self-distillation. This design transforms sparse-reward self-play into a dense-feedback co-evolution. Extensive experiments show that data-free π -Play surpasses fully supervised search agents and improves evolutionary efficiency by 2–3 $\times$ over conventional self-play. Code is available at <https://github.com/zhyaoch/pi-play>.

1 Introduction

Deep search agents leverage the reasoning capabilities of large language models (LLMs) and external search engines to perform multi-turn retrieval and analysis for complex questions, emerging as a promising paradigm for information acquisition (Shao et al., 2024; Guo et al., 2025; Zheng et al., 2025). Recent advances have shown that

reinforcement learning (RL) can substantially improve both reasoning and search behaviors, enabling LLM agents to tackle increasingly challenging information-seeking tasks (Shao et al., 2024; Guo et al., 2025). However, training strong search agents at scale remains fundamentally bottlenecked by data (Yue et al., 2026; Jin et al., 2025; Wu et al., 2025; Li et al., 2025; Gao et al., 2025). Supervised pipelines rely on labeled data and costly expert trajectories, while outcome-supervised RL often suffers from sparse rewards and poor credit assignment, especially in multi-turn search scenarios (Jin et al., 2025; Zhang et al., 2025a; Tu et al., 2026; Wang et al., 2025; Zhang et al., 2025b; Yue et al., 2025a).

A promising direction for alleviating data dependence is self-evolution through *self-play*. In existing self-play frameworks, models of the same scale alternately play the roles of examiner and student: the examiner autonomously constructs training tasks, while the student learns by solving those tasks (Yue et al., 2026; Huang et al., 2026; Lu et al., 2025; OpenAI et al., 2021; Chen et al., 2024). This paradigm allows the model to bootstrap its own curriculum without relying on manually collected training data, offering a promising route toward scalable and autonomous improvement. Despite its strengths, self-play still suffers from a critical limitation. The student is typically optimized only through sparse outcome rewards, which makes learning inefficient for multi-turn search tasks (Yue et al., 2026). Notably, self-play produces more than just the final training question-answer (QA) pair (q, o^*) . As shown in Fig. 2, self-play also naturally produces a high-quality yet previously overlooked intermediate artifact, the *question construction path* (QCP), denoted by c , which captures the multi-turn interaction process by which the examiner constructs the question through iterative search. Hence, the examiner’s output is more accurately represented as a triplet (q, c, o^*) , where

^{*}Work primarily done during an internship at Meituan.

[†]Corresponding author

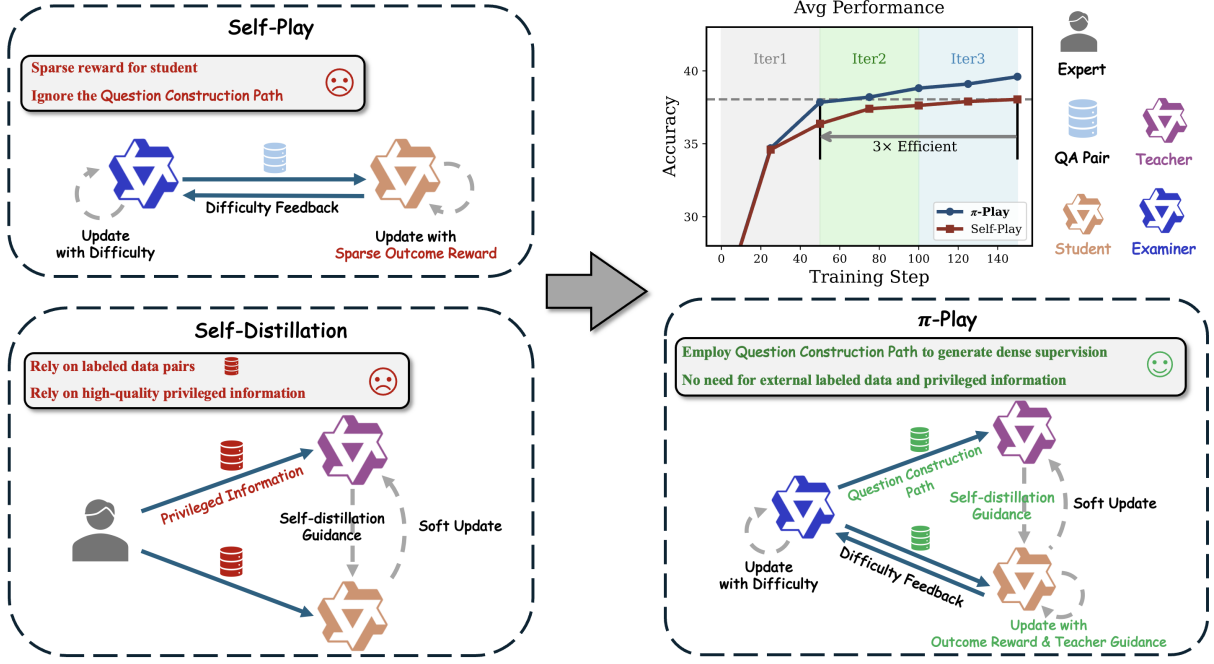


Figure 1: **Comparison of π -Play with other self-evolution frameworks.** All models (examiner, teacher, and student) in π -Play are initialized from the same base LLM and function as search agents. π -Play uses alternating optimization to evolve multiple agents in a closed loop. Compared to self-play, it overcomes the sparse-reward of the student and enables the student to be optimized under the joint effect of outcome rewards and teacher guidance.

the QCP records a reverse solution process from the answer back to the question. Rather than being merely an intermediate artifact, the QCP serves as a form of intrinsic privileged information for dense supervision. Unfortunately, existing self-play methods largely ignore this signal, since they cannot directly use it for supervised fine-tuning (SFT) (Yue et al., 2026; Huang et al., 2026; Lu et al., 2025; Fu et al., 2025), and thus fail to exploit it as a valuable source of dense supervision.

Another line of self-evolution, *self-distillation*, addresses the credit assignment problem by employing high-quality privileged information (Hübotter et al., 2026; Shenfeld et al., 2026; Zhao et al., 2026; Ye et al., 2026; Penaloza et al., 2026). Unlike on-policy distillation (Agarwal et al., 2024; Gu et al., 2024; Yue et al., 2025b; Lu and Lab, 2025), which relies on a larger external teacher, self-distillation uses a teacher model of the same scale as the student and augments it with privileged information to provide token-level dense supervision for the student. Common sources of such privileged information include expert demonstrations (Penaloza et al., 2026; Shenfeld et al., 2026), external (human) feedback (Hübotter et al., 2026; Wang et al., 2026), and prior knowledge (Ye et al., 2026; Sang et al., 2026). Prior studies have shown that such privileged supervision can significantly

enhance learning efficiency. However, obtaining high-quality privileged information is often non-trivial. In several prior works, privileged information is typically constructed with the help of human experts or stronger models (Shenfeld et al., 2026; Zhao et al., 2026; Ye et al., 2026). Furthermore, self-distillation typically relies on training data consisting of QA pairs during optimization. This dependence on both high-quality privileged information and curated training data makes self-distillation difficult to scale efficiently.

This work is motivated by a key observation: self-play naturally produces intrinsic privileged information that can be exploited for self-distillation. Since QCP records how the question is constructed from factual evidence, it provides privileged context that can help a same-scale teacher to generate more accurate rollouts than a student conditioned only on the question. Based on this insight, we propose *Privileged Information Self-Play* (π -Play), a multi-agent self-evolution framework in which an examiner first generates training tuples (q, c, o^*) , and a teacher model conditioned on the construction path c then provides token-level distillation signals to guide the student (Fig. 2). Through this mechanism, π -Play improves search and reasoning capability while transforming sparse-reward self-play into a dense-feedback self-

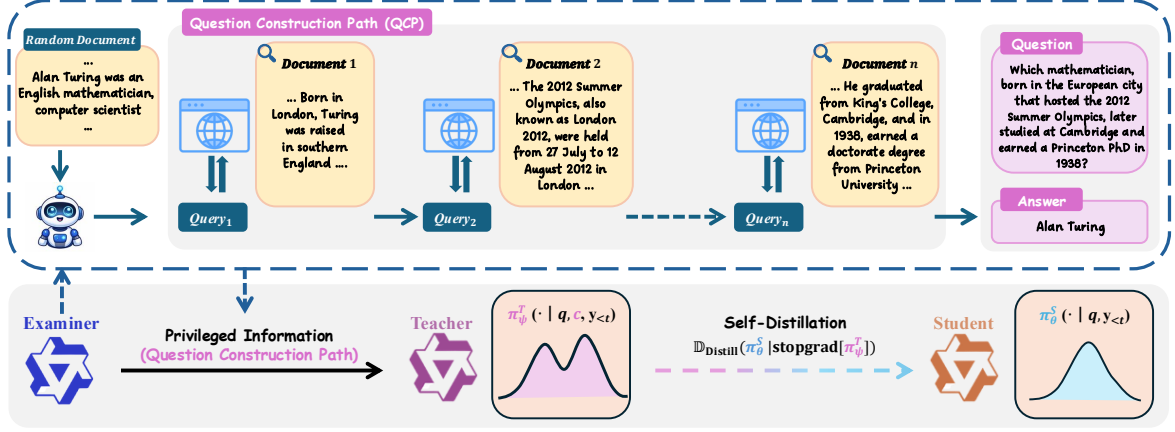


Figure 2: **Overview of QCP-guided self-distillation in π -Play.** The examiner is equipped with search tools and interacts with the search engine to obtain factual information, ensuring the correctness of both the synthesized QA pairs and their construction paths c . The teacher policy π_ψ^T leverages QCP as additional context to provide token-level supervision to the student policy π_θ^S along the student’s rollout y , by minimizing the per-token reverse KL divergence $\mathbb{D}_{\text{Distill}}(\pi_\theta^S \parallel \text{stopgrad}[\pi_\psi^T])$ (Eq. 10).

evolution loop. Through multiple efficient training iterations guided by the teacher, π -Play surpasses supervised baselines and conventional self-play baselines and exhibits stronger evolutionary efficiency (Fig.1). In summary, our main contributions are as follows:

- We propose π -Play, a novel multi-agent self-evolution framework that combines self-play and self-distillation, in which QCP bridges the examiner, teacher, and student, enabling their efficient co-evolution without external training data.
- We reveal that QCP is a new source of high-quality privileged information and that self-play can generate QCPs during task construction at low cost and at scale. This enables QCP-guided self-distillation without relying on human feedback or curated privileged data.
- We introduce a teacher role into self-play to transform QCPs into dense token-level supervision for the student through QCP-guided self-distillation. Extensive experiments show that π -Play surpasses fully supervised search agents and achieves $2\text{--}3\times$ higher evolutionary efficiency than conventional self-play.

2 Methods

2.1 Self-Play with Privileged Information

As shown in Fig. 1, we employ a self-evolution framework based on efficient multi-agent collaboration, in which all models function as search agents capable of leveraging external knowledge. Let y

denote a full response (or rollout), and $o(y)$ denote the final answer extracted from y ; for brevity, we write $o_i := o(y_i)$. The *examiner* π_ϕ^E , the *teacher* π_ψ^T , and the *student* π_θ^S are each optimized according to their respective objectives:

Optimization Objective

Examiner (Sec. 2.2): Learn to generate diverse and challenging questions.

$$\max_{\phi} \mathbb{E}_{(q, c, o^*) \sim \pi_{\phi}^E(\cdot), \{y_k\}_{k=1}^n \sim \pi_{\theta}^S(\cdot | q)} [r^d(o^*, \{o_k\}_{k=1}^n)], \quad (1)$$

Teacher (Sec. 2.4): Incorporate privileged information to generate more accurate rollouts while avoiding an excessive discrepancy between the teacher and the student.

$$\max_{\psi} \mathbb{E}_{(q, c, o^*) \sim \pi_{\phi}^E(\cdot), y \sim \pi_{\psi}^T(\cdot | q, c)} [\mathbb{I}(o = o^*) - \beta \mathbb{D}_{\text{KL}}(\pi_{\psi}^T(\cdot | q, c) \parallel \text{stopgrad}[\pi_{\theta}^S(\cdot | q)])], \quad (2)$$

Student (Sec. 2.3): Learns efficiently through a combination of outcome reward and teacher guidance, enhancing its evolutionary efficiency.

$$\max_{\theta} \mathbb{E}_{(q, c, o^*) \sim \pi_{\phi}^E(\cdot), y \sim \pi_{\theta}^S(\cdot | q)} [\mathbb{I}(o = o^*) - \lambda \mathbb{D}_{\text{Distill}}(\pi_{\theta}^S(\cdot | q) \parallel \text{stopgrad}[\pi_{\psi}^T(\cdot | q, c)])], \quad (3)$$

where r^d denotes the difficulty reward and $\mathbb{I}$ is the indicator function. To generate questions of moderate difficulty (i.e., suitable for the student’s current capability), the examiner’s reward r^d is defined over the distribution of predicted answers. If all predictions (i.e., $\{o_k\}_{k=1}^n$) are correct, the question

is considered trivial, whereas if none are correct, the question is likely too difficult for the student. To jointly optimize the examiner, teacher, and student, we adopt an alternating optimization loop that couples question generation, teacher guidance, and student improvement into a unified self-evolution process. As the student becomes stronger, it drives the examiner to generate increasingly challenging questions. Meanwhile, the student’s updated behavior is softly propagated to the teacher, allowing the teacher to track the student while remaining more stable. This iterative process establishes a continuously evolving curriculum. All three agents are initialized from the same base LLM and rely exclusively on the search tool to access external knowledge. Following prior work (Yue et al., 2026), we strictly adhere to a *training data-free setting*, avoiding any demonstrations, questions, or annotated answers from external sources or human experts.

2.2 Examiner Training

To utilize student feedback for training the examiner, we employ a difficulty reward function r^d that encourages both verifiability (the task must be solvable) and difficulty (the task must not be trivial) (Yue et al., 2026). Specifically, we leverage the student’s success rate on the generated questions as a proxy for these properties. Let k denote the number of correct solutions out of n sampled attempts. We penalize cases where the student either fails completely ($k = 0$) or succeeds trivially ($k = n$), thereby encouraging the examiner to generate questions of moderate difficulty. The reward is defined as:

$$r^d(o^*, \{o_i\}_{i=1}^n) = \mathbb{I}(0 < k < n) \frac{n-k}{n-1} + r^f, \quad (4)$$

where $k = \sum_{i=1}^n \mathbb{I}(o_i = o^*)$. The difficulty reward is maximized when exactly one solution is correct and decays linearly as the number of correct predictions increases. Here, r^f denotes a format reward that encourages the examiner to interleave reasoning with search during task generation, so that the synthesized questions are both factually grounded and accompanied by an informative construction path. These paths serve as privileged information, enabling the teacher to provide guidance to student.

Following prior work, we employ hop-grouped relative policy optimization (Yue et al., 2026) to train the examiner. Specifically, we estimate advantages by grouping structurally similar questions within a batch. Generated QA pairs are clustered according to their *cross-hop complexity*, measured by

the number of hops $h \in \mathcal{H}$. Intuitively, questions with fewer hops are typically simpler, whereas higher-hop questions demand extensive search and multi-turn reasoning. This hop-specific normalization of returns produces low-variance advantage estimates while avoiding the computational cost of sampling multiple candidate questions per prompt.

$$\mathcal{J}_{\text{Examiner}}(\phi) = \mathbb{E}_{\{(q_i, c_i, o_i^*) \sim \pi_\phi^E(\cdot), \{y_{i,k}\}_{k=1}^n \sim \pi_\theta^S(\cdot | q_i)\}_{i=1}^N} \left[\frac{1}{N} \sum_{h \in \mathcal{H}} \sum_{i \in \mathcal{I}_h} \log \pi_\phi^E A_{i,h} - \beta \mathbb{D}_{\text{KL}}(\pi_\phi^E \| \pi_{\text{ref}}) \right], \quad (5)$$

where N denotes the batch size and β controls the strength of the KL regularization. The advantage of each generated QA triplet (q_i, c_i, o_i^*) is obtained by hop-wise reward normalization:

$$A_{i,h} = \frac{r_i^d - \mathbb{E}_{j \in \mathcal{I}_h} [r_j^d]}{\sqrt{\text{Var}_{j \in \mathcal{I}_h} [r_j^d]} + \delta} \quad (6)$$

where $\mathcal{I}_h$ is the set of questions in the hop group h , and δ is a small constant for numerical stability.

2.3 Student Training

For student training, we sample training tuples (q, c, o^*) from the examiner policy π_ϕ^E . The student policy π_θ^S then generates candidate rollouts for each question and is optimized using a hybrid training objective. Importantly, the construction path c is only visible to the teacher model, ensuring that the teacher can produce reliable guidance while the student learns to solve the task without direct access to privileged information. Formally, the student objective is defined as follows:

$$\mathcal{J}_{\text{play}}(\theta) = \mathbb{E}_{(q, c, o^*) \sim \pi_\phi^E, \{y_i\}_{i=1}^G \sim \pi_\theta^S(\cdot | q)} \left[\underbrace{\frac{1}{G} \sum_{i=1}^G \sum_{t=1}^{|y_i|} \mathcal{L}_{i,t}}_{\text{Learning from outcome reward}} - \underbrace{\beta \mathbb{D}_{\text{KL}}(\pi_\theta^S \| \pi_{\text{ref}}) - \lambda \mathbb{D}_{\text{Distill}}(\pi_\theta^S \| \pi_\psi^T)}_{\text{Teacher guidance}} \right], \quad (7)$$

$$\mathcal{L}_{i,t} = \min(w_{i,t} A_i, \text{clip}(w_{i,t}, 1 - \epsilon, 1 + \epsilon) A_i), \quad (8)$$

The overall objective consists of two complementary components. The first term corresponds to group relative policy optimization (GRPO), which improves the student policy using outcome rewards derived from answer correctness (i.e., $r^e(q, o_i) = \mathbb{I}(o_i = o^*)$). The importance weight $w_{i,t}$ and normalized advantage A_i are given by:

$$w_{i,t} = \frac{\pi_\theta^S(y_{i,t} | q, y_{i,<t})}{\pi_{\theta_{\text{old}}}^S(y_{i,t} | q, y_{i,<t})}, \quad A_i = \frac{r_i^e - \mathbb{E}_{j \in G} [r_j^e]}{\sqrt{\text{Var}_{j \in G} [r_j^e]} + \delta} \quad (9)$$

By computing advantages from group statistics, GRPO reinforces successful rollouts while penalizing failed ones. The second term is a self-distillation objective, where the student policy is aligned with the teacher policy π_ψ^T along student’s rollout. The distillation loss is defined as follows:

$$\mathbb{D}_{\text{Distill}}(\pi_\theta^S \parallel \pi_\psi^T) = \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \text{KL}(\pi_\theta^S(\cdot \mid q, y_{i,<t}) \parallel \text{stopgrad}(\pi_\psi^T(\cdot \mid q, \mathbf{c}, y_{i,<t}))) \quad (10)$$

The full formulation of the distillation loss is provided in Appendix D. Since the teacher has access to privileged information in the form of the QCP, it can provide reliable token-level guidance. This dense supervision complements sparse outcome rewards through a favorable bias-variance trade-off (Schulman et al., 2016; Gu et al., 2016): outcome rewards are unbiased but high-variance, whereas teacher guidance may introduce modest bias while substantially reducing variance. Their combination enables more efficient credit assignment and faster policy improvement.

2.4 Teacher Updating

Although Eq. (2) defines the ideal teacher objective, directly optimizing it would introduce additional computational overhead. In practice, to provide stable teacher supervision while allowing the teacher to co-evolve with the student, we approximate this objective by updating the teacher parameters as an exponential moving average (EMA) of the student parameters (Hübotter et al., 2026; Penaloza et al., 2026):

$$\psi \leftarrow (1 - \tau)\psi + \tau\theta, \quad \tau \in (0, 1), \quad (11)$$

where τ controls the teacher update rate. This update keeps the teacher relatively stable while enabling it to gradually track the student’s improvements over training.

2.5 The Co-Evolution Procedure of π -Play

In summary, we present π -Play, a data-free self-evolution framework that jointly optimizes the examiner, teacher, and student models (Fig. 1). In each iteration, the examiner generates QA pairs together with QCP, and is trained via student-derived difficulty feedback to produce challenging yet solvable questions. The student improves its search and reasoning abilities through training on QA pairs generated by examiner, while the teacher leverages the QCP as additional context to provide token-level guidance to the student. After each update,

Algorithm 1 The Evolution Process of π -Play

Require: Examiner model π_ϕ^E , Teacher model π_ψ^T , Student model π_θ^S , Base LLM π_{ref}

- 1: Initialize π_ϕ^E , π_ψ^T , and π_θ^S from Base LLM π_{ref}
- 2: **for** iteration $j \leftarrow 1$ to M **do**
- 3: *# Updating Examiner Model*
- 4: **for** step $k \leftarrow 1$ to W **do**
- 5: Sample N QA triplets $\{(q_i, c_i, o_i^*)\}_{i=1}^N$ from the examiner policy π_ϕ^E
- 6: Update $\pi_\phi^E(\cdot)$ with Eq. (5)
- 7: **end for**
- 8:
- 9: *# Generate Training Data For Student*
- 10: Generate $\mathcal{D} = \{(q_i, c_i, o_i^*)\}_{i=1}^{N_D}$ from $\pi_\phi^E(\cdot)$
- 11:
- 12: *# Updating Student and Teacher Model*
- 13: **for** step $k \leftarrow 1$ to W **do**
- 14: Sample a batch $\mathcal{D}_b$ from $\mathcal{D}$
- 15: **for** each QA triplets (q, c, o^*) in $\mathcal{D}_b$ **do**
- 16: Sample G rollouts $\{y_i\}_{i=1}^G$ from the student policy $\pi_\theta^S(\cdot \mid q)$
- 17: **end for**
- 18: Update π_θ^S with Eq. (7) by teacher guidance; **Then** Soft Update π_ψ^T with Eq. (11)
- 19: **end for**
- 20: **end for**
- 21: **return** Student model π_θ^S

the teacher is softly aligned with the student, enabling both models to co-evolve. This alternating optimization loop forms a symbiotic feedback cycle in which stronger students drive the examiner toward more challenging questions, while teacher guidance accelerates student improvement. The training process is summarized in Algorithm 1.

3 Experiments

3.1 Setup

Datasets & Models. We conduct experiments on three models from the Qwen-3 series (Yang et al., 2025). The base models are Qwen3-4B, Qwen3-4B-Instruct, and Qwen3-8B. More details on experimental setups and training procedures can be found in Appendix B. We evaluate π -Play primarily on three one-hop benchmarks NQ (Kwiatkowski et al., 2019), TriviaQA (Joshi et al., 2017), PopQA (Mallen et al., 2023), as well as four multi-hop QA benchmarks, including HotpotQA (Yang et al.,

Table 1: **The main results of π -Play.** **Bold** values indicate the best result; underline values denote the second-best. Through efficient teacher guidance, π -Play outperforms both supervised and self-play search agents.

	General QA			Multi-Hop QA				Total
	NQ	TriviaQA	PopQA	HotpotQA	2WikiMQA	MuSiQue	Bamboogle	
<i>Qwen3-4B</i>								
ReAct	7.2	16.6	10.3	11.2	12.7	1.6	8.8	68.4
Search-R1	35.2	57.3	39.5	30.7	28.9	10.5	37.6	239.7
SQLM*	<u>39.9</u>	<u>60.4</u>	42.1	33.0	<u>30.8</u>	<u>10.8</u>	29.6	<u>246.6</u>
Dr.Zero	40.4	61.2	44.3	31.0	29.3	7.8	31.2	245.2
π -Play	38.1	58.4	<u>42.3</u>	<u>32.3</u>	32.5	11.2	<u>35.2</u>	250.0
<i>Qwen3-4B-Instruct</i>								
ReAct	18.4	45.5	28.4	22.8	18.0	6.3	27.2	166.6
Search-R1	39.5	<u>62.6</u>	41.3	34.9	24.8	12.9	50.0	266.0
SQLM*	39.0	<u>62.2</u>	<u>41.9</u>	34.1	27.3	10.2	35.2	249.9
Dr.Zero	<u>41.3</u>	61.6	44.0	<u>38.4</u>	<u>29.8</u>	14.4	36.8	<u>266.3</u>
π -Play	41.8	63.2	44.0	38.5	32.3	<u>13.4</u>	<u>44.0</u>	277.2
<i>Qwen3-8B</i>								
ReAct	15.9	37.3	14.9	22.3	22.5	7.4	28.8	149.1
Search-R1	35.7	59.9	40.0	32.0	27.8	9.8	37.6	242.8
ToolForge	30.3	16.8	35.2	-	-	37.8	48.0	-
SQLM*	38.9	60.5	42.7	33.1	31.7	9.9	32.0	248.8
Dr.Zero	<u>42.1</u>	<u>62.5</u>	<u>45.8</u>	<u>36.6</u>	34.8	<u>13.1</u>	<u>40.0</u>	<u>274.9</u>
π -Play	43.0	64.6	47.2	38.9	<u>34.2</u>	12.4	<u>40.0</u>	280.3

2018), 2WikiMQA (Ho et al., 2020), MuSiQue (Trivedi et al., 2022), and Bamboogle (Press et al., 2023).

Baselines & Evaluation. To demonstrate the efficacy of π -Play, we compare it against a variety of baseline search agents: (1) *training-free*: ReAct (Yao et al., 2023); (2) *supervised RL*: Search-R1 (Jin et al., 2025) and ToolForge (Chen et al., 2025a), and (3) *self-play*: Dr.Zero (Yue et al., 2026) and SQLM* (Chen et al., 2025b; Yue et al., 2026). All models are evaluated using the exact match scores with identical search engine (E5-base (Wang et al., 2022)) and corpus settings (English Wikipedia dump (Karpukhin et al., 2020)). We report the performance of the checkpoint from its best-performing training iteration (step).

3.2 Main Results

We first analyze the main evaluation results as reported in Table 1 and derive several key observations from them: (1) *Strong Overall Performance*: π -Play achieves substantial improvements in overall performance over base LLM (e.g., ReAct), demonstrating strong robustness and generalization across diverse task types and model scales.

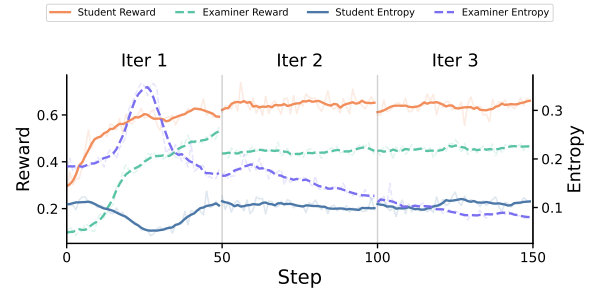


Figure 3: **Iterative reward and entropy dynamics of the examiner and student in π -Play with Qwen3-4B-Instruct.** Both reward and entropy reach a converged state by Iteration 3.

The results further demonstrate the effectiveness and superiority of the multi-agent self-evolution framework for search agents. (2) *π -Play surpasses supervised RL methods.* π -Play delivers strong performance without using any training data. In terms of average performance, it surpasses the Search-R1 by 6.3%, 4.2%, and 15.4% on Qwen3-4B, Qwen3-4B-Instruct, and Qwen3-8B, respectively. This self-evolution framework demonstrates greater performance gains when instantiated with a stronger base LLM. (3) *π -Play surpasses self-play methods.* π -

Table 2: Learning dynamics of π -Play with increasing iterations.

	Methods	General QA			Multi-Hop QA				Total
		NQ	TriviaQA	PopQA	HotpotQA	2WikiMQA	MuSiQue	Bamboogle	
Qwen3-4B									
Iter1	Dr.Zero	36.1	58.7	42.8	28.6	25.9	6.2	21.6	219.9
	π -Play	38.6	59.4	43.5	32.2	31.4	10.7	32.8	248.6
Iter2	Dr.Zero	40.6	60.5	44.4	30.8	29.1	8.0	28.8	242.2
	π -Play	38.1	58.4	42.3	32.3	32.5	11.2	35.2	250.0
Iter3	Dr.Zero	40.4	61.2	44.3	31.0	29.3	7.8	31.2	245.2
	π -Play	41.1	62.0	44.8	32.5	31.5	9.0	28.8	249.7
Qwen3-4B-Instruct									
Iter1	Dr.Zero	39.8	61.7	42.6	35.4	27.4	11.7	36.0	254.6
	π -Play	40.7	62.1	42.9	36.7	30.1	13.2	39.2	264.9
Iter2	Dr.Zero	39.3	61.7	43.2	38.2	28.4	14.2	38.4	263.4
	π -Play	41.2	62.5	43.5	37.8	32.5	14.2	40.0	271.7
Iter3	Dr.Zero	41.3	61.6	44.0	38.4	29.8	14.4	36.8	266.3
	π -Play	41.8	63.2	44.0	38.5	32.3	13.4	44.0	277.2
Qwen3-8B									
Iter1	Dr.Zero	36.7	59.5	42.9	32.1	26.9	9.5	18.4	226.0
	π -Play	38.2	59.7	43.4	34.1	33.2	10.9	35.2	254.7
Iter2	Dr.Zero	36.7	60.0	43.2	36.6	35.4	12.7	38.4	263.0
	π -Play	42.9	63.2	46.4	37.7	34.7	12.4	36.0	273.3
Iter3	Dr.Zero	42.1	62.5	45.8	36.6	34.8	13.1	40.0	274.9
	π -Play	43.0	64.6	47.2	38.9	34.2	12.4	40.0	280.3

Play outperforms the self-play methods (SQLM* and Dr.Zero) across multiple model scales, benefiting from the additional guidance provided by the teacher to the student. Notably, the performance gains are even more substantial on multi-hop benchmarks, where complex multi-step reasoning is required. We attribute this advantage to token-level credit assignment from the teacher, which provides finer-grained and effective supervision for long-horizon reasoning.

3.3 Training Dynamics

To better understand the self-evolving dynamics of π -Play, we analyze model performance across training iterations, with detailed comparisons with Dr.Zero summarized in Table 2. These results lead to several key observations: (1) Across all three iterations, the student model shows a steady upward performance trend, consistently outperforming Dr.Zero after every iteration. This highlights the effectiveness of the examiner-teacher-student interplay and suggests that collaborative self-evolution is more effective than standard self-play by enabling more efficient information sharing among agents. (2) After the first training iteration, the student in π -Play achieves substantial gains in both

search and reasoning abilities. In terms of performance, it already matches or even surpasses the converged performance of Dr.Zero after three iterations, demonstrating the superior evolutionary efficiency. (3) After the second iteration, the improvement trends begin to vary across model sizes. While Qwen3-4B-Instruct and Qwen3-8B continue to show modest gains, Qwen3-4B drops slightly from 250 to 249.7, suggesting that performance has started to plateau. Beyond this point, further iterations bring only marginal or no additional improvements across model sizes. In summary, these dynamics validate the design of π -Play, demonstrating that the introduction of the privileged teacher brings significant benefits.

4 Further Analysis

In this section, we analyze the importance of the QCP, the co-evolution dynamic of the examiner, and the search behavior of the student. Further experiments, including extensive ablation studies, training cost are provided in Appendix C.

Table 3: **Ablation study on the question construction path (QCP) with Qwen3-4B-Instruct.** π -Play w/o Distillation denotes the variant of π -Play without teacher-guided distillation loss. Variants of the form π -Play w/ [Privileged Info] replace QCP with different forms of privileged information for self-distillation.

	General QA			Multi-Hop QA				Total
	NQ	TriviaQA	PopQA	HotpotQA	2WikiMQA	MuSiQue	Bamboogle	
π -Play w/o Distillation	41.2	62.3	43.6	37.7	29.0	13.3	38.4	265.5
π -Play w/ GT	42.0	62.0	44.5	37.7	29.4	13.3	35.2	264.1
π -Play w/ GT+Hop	42.1	61.9	42.9	37.7	29.1	14.0	39.2	266.9
π -Play w/ Partial QCP	41.6	62.1	43.3	38.3	31.0	13.6	43.2	273.1
π -Play w/ QCP (Ours)	41.8	63.2	44.0	38.5	32.3	13.4	44.0	277.2

4.1 Ablation Study on the Question Construction Path

As shown in Table 3, we further evaluate the effectiveness of QCP as privileged information by comparing it with several alternative forms of teacher-side privileged context, including the ground-truth answer alone (π -Play w/ GT), the ground-truth answer combined with the question hop count (π -Play w/ GT+HOP), and a partial QCP obtained by randomly truncating half of the original QCP (π -Play w/ Partial QCP). Among these variants, using GT yields the worst performance, which is nearly comparable to the variant without a teacher model (π -Play w/o Distillation). This is because GT contains only the final answer and provides little information about the underlying logic used to construct the question, making it difficult for the teacher to provide effective guidance to the student. In contrast, the full QCP achieves the best performance, while Partial QCP performs second best. Moreover, Partial QCP still substantially outperforms both GT and GT+HOP, further highlighting the effectiveness of QCP as privileged information.

Table 4: **Co-evolutionary dynamic of the examiner.** We report the student’s accuracy on the datasets generated by the examiner at different steps.

	Performance of Evaluated Model		
	Base Model	Student(step 50)	Student(step 100)
$\mathcal{D}_{\text{step 50}}$	27.5	57.1	58.4
$\mathcal{D}_{\text{step 100}}$	27.0	51.9	57.3
$\mathcal{D}_{\text{step 150}}$	22.9	45.1	53.3

4.2 Evolution of Question Difficulty

To understand the co-evolutionary dynamic of the examiner, we examined how the tasks it generated changed across iterations. After each of the training iterations, we sampled 2000 questions from its policy, creating three distinct evaluation sets: $\mathcal{D}_{\text{step 50}}$, $\mathcal{D}_{\text{step 100}}$, and $\mathcal{D}_{\text{step 150}}$. As shown in Table 4, the

examiner generates progressively more challenging questions as training proceeds. This is evidenced by the performance of a fixed solver on these evolving question sets: for instance, the static student (Step 50) drops from 57.1 on $\mathcal{D}_{\text{step 50}}$ to 45.1 on $\mathcal{D}_{\text{step 150}}$. This suggests that examiner successfully increases task difficulty over the course of training.

4.3 Search Behavior Analysis

To assess the effect of QCP-guided self-distillation on search behavior, we quantitatively analyze search count and query redundancy across seven QA benchmarks. As shown in Table 5, compared with conventional self-play methods such as SQLM* and Dr.Zero, π -Play achieves higher accuracy with fewer search actions, indicating more efficient behavior. The same trend holds for query redundancy. Fine-grained supervision leads to lower redundancy and more effective queries.

Table 5: **Quantitative analysis of search count and query redundancy with Qwen3-4B-Instruct.** We report the average number of search actions and the average query redundancy across seven QA benchmarks (NQ, TriviaQA, PopQA, HotpotQA, 2WikiMQA, MuSiQue, and Bamboogle).

	Accuracy $\uparrow$	Search Count $\downarrow$	Query Redundancy $\downarrow$
SQLM*	35.7	2.3	0.43
Dr.Zero	38.0	2.4	0.48
π -Play	39.6	1.9	0.37

5 Conclusion

In this work, we propose π -Play, a novel self-evolution framework that automatically generates training tasks through self-play, which provides high-quality privileged information for self-distillation at low cost and at scale. Our key insight is that the QCP generated during self-play constitutes an intrinsic form of privileged information

that can be transformed into dense token-level supervision through self-distillation. Extensive experiments demonstrate that π -Play outperforms fully supervised search agents and enables more efficient LLM self-evolution with no need for external labels or a stronger teacher.

6 Limitations

While π -Play provides a new framework for multi-agent self-evolution, our current experiments instantiate it with a foundational self-distillation pipeline. This design helps measure the effectiveness of the QCP-guided self-distillation, but it may not fully exploit recent advances in self-distillation. Consequently, the evolutionary efficiency and supervision quality of π -Play could be further enhanced by incorporating more advanced on-policy distillation and self-distillation techniques. Exploring such integrations is an important direction for future work.

References

- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos Garea, Matthieu Geist, and Olivier Bachem. 2024. On-policy distillation of language models: Learning from self-generated mistakes. In *The Twelfth International Conference on Learning Representations*, pages 1–18, Vienna, Austria. OpenReview.net.
- Hao Chen, Zhexin Hu, Jiajun Chai, Haocheng Yang, Hang He, Xiaohan Wang, Wei Lin, Luhang Wang, Guojun Yin, and Zhuofeng Zhao. 2025a. Toolforge: A data synthesis pipeline for multi-hop search without real-world apis. *arXiv preprint arXiv:2512.16149*.
- Lili Chen, Mihir Prabhudesai, Katerina Fragkiadaki, Hao Liu, and Deepak Pathak. 2025b. Self-questioning language models. *arXiv preprint arXiv:2508.03682*.
- Zixiang Chen, Yihe Deng, Huizhuo Yuan, Kaixuan Ji, and Quanquan Gu. 2024. Self-play fine-tuning converts weak language models to strong language models. *arXiv preprint arXiv:2401.01335*.
- Yuqian Fu, Tinghong Chen, Jiajun Chai, Xihuai Wang, Songjun Tu, Guojun Yin, Wei Lin, Qichao Zhang, Yuanheng Zhu, and Dongbin Zhao. 2025. Srft: A single-stage method with supervised and reinforcement fine-tuning for reasoning. *arXiv preprint arXiv:2506.19767*.
- Jiaxuan Gao, Wei Fu, Minyang Xie, Shusheng Xu, Chuyi He, Zhiyu Mei, Banghua Zhu, and Yi Wu. 2025. Beyond ten turns: Unlocking long-horizon agentic search with large-scale asynchronous rl. *arXiv preprint arXiv:2508.07976*.
- Shixiang Gu, Timothy Lillicrap, Zoubin Ghahramani, Richard E Turner, and Sergey Levine. 2016. Q-prop: Sample-efficient policy gradient with an off-policy critic. *arXiv preprint arXiv:1611.02247*.
- Yuxian Gu, Li Dong, Furu Wei, and Minlie Huang. 2024. MiniLLM: Knowledge distillation of large language models. In *The Twelfth International Conference on Learning Representations*, pages 1–24, Vienna, Austria. OpenReview.net.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shitong Ma, Peiyi Wang, Xiao Bi, and 1 others. 2025. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*.
- Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing a multi-hop QA dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 6609–6625, Barcelona, Spain (Online). International Committee on Computational Linguistics.
- Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Li, Jiaxin Huang, Haitao Mi, and Dong Yu. 2026. R-zero: Self-evolving reasoning llm from zero data. *arXiv preprint arXiv:2508.05004*.
- Jonas Hübötter, Frederike Lübeck, Lejs Behric, Anton Baumann, Marco Bagatella, Daniel Marta, Ido Hakimi, Idan Shenfeld, Thomas Kleine Buening, Carlos Guestrin, and Andreas Krause. 2026. Reinforcement learning via self-distillation. *arXiv preprint arXiv:2601.20802*.
- Bowen Jin, Hansi Zeng, Zhenrui Yue, Jinsung Yoon, Serkan O Arik, Dong Wang, Hamed Zamani, and Jiawei Han. 2025. Search-r1: Training LLMs to reason and leverage search engines with reinforcement learning. In *Second Conference on Language Modeling*.
- Mandar Joshi, Eunsol Choi, Daniel S. Weld, and Luke Zettlemoyer. 2017. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, pages 1601–1611, Vancouver, Canada. Association for Computational Linguistics.
- Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick SH Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pages 6769–6781, Online. Association for Computational Linguistics.
- Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew

- Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466.
- Kuan Li, Zhongwang Zhang, Huifeng Yin, Liwen Zhang, Litu Ou, Jialong Wu, Wenbiao Yin, Baixuan Li, Zhengwei Tao, Xinyu Wang, Weizhou Shen, Junkai Zhang, Dingchu Zhang, Xixi Wu, Yong Jiang, Ming Yan, Pengjun Xie, Fei Huang, and Jingren Zhou. 2025. Websailor: Navigating superhuman reasoning for web agent. *arXiv preprint arXiv:2507.02592*.
- Bo Liu, Leon Guertler, Simon Yu, Zichen Liu, Penghui Qi, Daniel Balcells, Mickel Liu, Cheston Tan, Weiyan Shi, Min Lin, Wee Sun Lee, and Natasha Jaques. 2026. Spiral: Self-play on zero-sum games incentivizes reasoning via multi-agent multi-turn reinforcement learning. *arXiv preprint arXiv:2506.24119*.
- Bo Liu, Chuanyang Jin, Seungone Kim, Weizhe Yuan, Wenting Zhao, Iliia Kulikov, Xian Li, Sainbayar Sukhbaatar, Jack Lanchantin, and Jason Weston. 2025. Spice: Self-play in corpus environments improves reasoning. *arXiv preprint arXiv:2510.24684*.
- Hongliang Lu, Yuhang Wen, Pengyu Cheng, Ruijin Ding, Jiaqi Guo, Haotian Xu, Chutian Wang, Haonan Chen, Xiaoxi Jiang, and Guanjun Jiang. 2025. Search self-play: Pushing the frontier of agent capability without supervision. *arXiv preprint arXiv:2510.18821*.
- Kevin Lu and Thinking Machines Lab. 2025. [On-policy distillation](#). Thinking Machines Lab: Connectionism.
- Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics*, pages 9802–9822, Toronto, Canada. Association for Computational Linguistics.
- OpenAI OpenAI, Matthias Plappert, Raul Sampeiro, Tao Xu, Ilge Akkaya, Vineet Kosaraju, Peter Welinder, Ruben D’Sa, Arthur Petron, Henrique P. d. O. Pinto, Alex Paino, Hyeonwoo Noh, Lilian Weng, Qiming Yuan, Casey Chu, and Wojciech Zaremba. 2021. Asymmetric self-play for automatic goal discovery in robotic manipulation. *arXiv preprint arXiv:2101.04882*.
- Emiliano Penaloza, Dheeraj Vattikonda, Nicolas Gontier, Alexandre Lacoste, Laurent Charlin, and Massimo Caccia. 2026. Privileged information distillation for language models. *arXiv preprint arXiv:2602.04942*.
- Ofir Press, Muru Zhang, Sewon Min, Ludwig Schmidt, Noah A Smith, and Mike Lewis. 2023. Measuring and narrowing the compositionality gap in language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 5687–5711, Singapore. Association for Computational Linguistics.
- Hejian Sang, Yuanda Xu, Zhengze Zhou, Ran He, Zhipeng Wang, and Jiachen Sun. 2026. Crisp: Compressed reasoning via iterative self-policy distillation. *arXiv preprint arXiv:2603.05433*.
- John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. 2016. High-dimensional continuous control using generalized advantage estimation. In *The Fourth International Conference on Learning Representations*, pages 1–14, San Juan, Puerto Rico. OpenReview.net.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, and 1 others. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*.
- Idan Shenfeld, Mehul Damani, Jonas Hübötter, and Pulkit Agrawal. 2026. Self-distillation enables continual learning. *arXiv preprint arXiv:2601.19897*.
- Huatong Song, Jinhao Jiang, Yingqian Min, Jie Chen, Zhipeng Chen, Wayne Xin Zhao, Lei Fang, and Jirong Wen. 2025. R1-searcher: Incentivizing the search capability in llms via reinforcement learning. *arXiv preprint arXiv:2503.05592*.
- Hao Sun, Zile Qiao, Jiayan Guo, Xuanbo Fan, Yingyan Hou, Yong Jiang, Pengjun Xie, Yan Zhang, Fei Huang, and Jingren Zhou. 2025. Zeroscore: Incentivize the search capability of llms without searching. *arXiv preprint arXiv:2505.04588*.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. Musique: Multi-hop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10:539–554.
- Songjun Tu, Chengdong Xu, Qichao Zhang, Yaocheng Zhang, Xiangyuan Lan, Linjing Li, and Dongbin Zhao. 2026. Dynamic dual-granularity skill bank for agentic rl. *arXiv preprint arXiv:2603.28716*.
- Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Text embeddings by weakly-supervised contrastive pre-training. *arXiv preprint arXiv:2212.03533*.
- Yinjie Wang, Xuyang Chen, Xiaolong Jin, Mengdi Wang, and Ling Yang. 2026. Openclaw-rl: Train any agent simply by talking. *arXiv preprint arXiv:2603.10165*.
- Ziliang Wang, Xuhui Zheng, Kang An, Cijun Ouyang, Jialu Cai, Yuhang Wang, and Yichao Wu. 2025.

- Stepsearch: Igniting llms search ability via step-wise proximal policy optimization. *arXiv preprint arXiv:2505.15107*.
- Jialong Wu, Baixuan Li, Runnan Fang, Wenbiao Yin, Liwen Zhang, Zhenglin Wang, Zhengwei Tao, Ding-Chu Zhang, Zekun Xi, Xiangru Tang, Yong Jiang, Pengjun Xie, Fei Huang, and Jingren Zhou. 2025. Webdancer: Towards autonomous information seeking agency. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, pages 1–29, San Diego, USA. Curran Associates Inc.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, and 41 others. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, Brussels, Belgium. Association for Computational Linguistics.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2023. React: Synergizing reasoning and acting in language models. In *The Eleventh International Conference on Learning Representations*, pages 1–33, Kigali, Rwanda. OpenReview.net.
- Tianzhu Ye, Li Dong, Xun Wu, Shaohan Huang, and Furu Wei. 2026. On-policy context distillation for language models. *arXiv preprint arXiv:2602.12275*.
- Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Xian Li, Sainbayar Sukhbaatar, Jing Xu, and Jason E Weston. 2024. Self-rewarding language models. In *Proceedings of the 41st International Conference on Machine Learning*, pages 57905–57923, Vienna, Austria. PMLR.
- Chuhuai Yue, Chengqi Dong, Yinan Gao, Hang He, Jiajun Chai, Guojun Yin, and Wei Lin. 2025a. Promoting efficient reasoning with verifiable stepwise reward. *arXiv preprint arXiv:2508.10293*.
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. 2025b. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model? In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, pages 1–36, San Diego, USA. Curran Associates Inc.
- Zhenrui Yue, Kartikeya Upasani, Xianjun Yang, Suyu Ge, Shaoliang Nie, Yuning Mao, Zhe Liu, and Dong Wang. 2026. Dr. zero: Self-evolving search agents without training data. *arXiv preprint arXiv:2601.07055*.
- Yaocheng Zhang, Haohuan Huang, Zijun Song, Yuanheng Zhu, Qichao Zhang, Zijie Zhao, and Dongbin Zhao. 2025a. Criticsearch: Fine-grained credit assignment for search agents via a retrospective critic. *arXiv preprint arXiv:2511.12159*.
- Yaocheng Zhang, Yuanheng Zhu, Yuqian Fu, Songjun Tu, and Dongbin Zhao. 2025b. Offline goal-conditioned reinforcement learning with elastic-subgoal diffused policy learning. In *Proceedings of the 24th International Conference on Autonomous Agents and Multiagent Systems*, page 2336–2344, Richland, SC. International Foundation for Autonomous Agents and Multiagent Systems.
- Siyan Zhao, Zhihui Xie, Mengchen Liu, Jing Huang, Guan Pang, Feiyu Chen, and Aditya Grover. 2026. Self-distilled reasoner: On-policy self-distillation for large language models. *arXiv preprint arXiv:2601.18734*.
- Yuxiang Zheng, Dayuan Fu, Xiangkun Hu, Xiaojie Cai, Lyumanshan Ye, Pengrui Lu, and Pengfei Liu. 2025. DeepResearcher: Scaling deep research via reinforcement learning in real-world environments. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 414–431, Suzhou, China. Association for Computational Linguistics.

A Related Work

A.1 Deep Search Agents

Deep search agents leverage the reasoning capabilities of large language models and external search engines to perform multi-turn retrieval and analysis for complex questions, emerging as a promising paradigm for information acquisition. Recent work has leveraged RL to further enhance both reasoning and access to up-to-date knowledge, enabling LLMs to tackle complex tasks more effectively (Shao et al., 2024; Guo et al., 2025). Some agentic RL works, including Search-R1 (Jin et al., 2025), R1-Searcher (Song et al., 2025), DeepResearcher (Zheng et al., 2025), and ZeroSearch (Sun et al., 2025), further enhance question-answering capabilities but remain constrained by limited training data. To scale agentic RL, some pipelines (Wu et al., 2025; Li et al., 2025; Gao et al., 2025) employ offline question-synthesis strategies, yet they do not explicitly couple task generation with the evolving capability of the solver. In contrast, self-play enables search agents to jointly generate and solve tasks without human annotations, reducing reliance on manual supervision and allowing agentic RL to scale to broader scenarios (Huang et al., 2026; Lu et al., 2025; Yue et al., 2026).

A.2 Self-play for LLMs

Self-play enables LLMs to autonomously improve their reasoning and problem-solving capabilities by iteratively generating tasks and learning from their own experiences (Liu et al., 2026; Huang et al., 2026; Lu et al., 2025; Liu et al., 2025; Yue et al., 2026). Early approaches leverage the LLM as both generator and evaluator, refining its policy without human supervision (OpenAI et al., 2021; Chen et al., 2024). For instance, self-rewarding LLMs employ iterative training loops where the model judges its own outputs to construct preference data for optimization (Yuan et al., 2024). More recent frameworks, such as R-Zero (Huang et al., 2026), SSP (Lu et al., 2025), and Dr.Zero (Yue et al., 2026), typically involve only two roles: an *examiner*, which generates QA pairs, and a *student*, which is optimized via outcome-based RL on these self-generated tasks. These methods typically rely on sparse outcome rewards and do not exploit the construction paths produced during question generation. Such coarse-grained feedback makes it difficult to distinguish effective from ineffective tool-use behaviors, leading to inefficient credit as-

signment and slow policy improvement. In contrast, π -Play incorporates these construction paths into student optimization as token-level supervision.

A.3 Privileged Information Self-distillation for LLMs

Self-distillation is a training paradigm that enables a student model to improve by learning from its own generated outputs. In this process, a teacher model evaluates students’ rollouts and provides supervision signals at the token level to guide students. Unlike on-policy distillation (Agarwal et al., 2024; Gu et al., 2024; Yue et al., 2025b; Lu and Lab, 2025), self-distillation does not rely on a larger external teacher. Instead, the teacher typically shares the same architecture and scale as the student, but is augmented with privileged information to provide reliable supervision (Hübotter et al., 2026; Shenfeld et al., 2026; Zhao et al., 2026; Ye et al., 2026; Penaloza et al., 2026). Such dense supervision has been shown to effectively enhance the student model’s learning efficiency (Hübotter et al., 2026; Shenfeld et al., 2026; Ye et al., 2026; Penaloza et al., 2026). However, obtaining high-quality privileged information is often nontrivial. In several prior works, privileged information is typically constructed with the help of human experts or stronger models (Shenfeld et al., 2026; Zhao et al., 2026; Ye et al., 2026), including expert demonstrations (Penaloza et al., 2026; Shenfeld et al., 2026) and prior knowledge (Ye et al., 2026; Sang et al., 2026), which limits the scalability of self-distillation. In π -Play, the privileged signal provided to the teacher model is derived from the task-construction process in self-play, thereby avoiding dependence on externally provided privileged information.

B Implementation

In our experiments, we implement π -Play through alternating optimization over the examiner and the student-teacher modules. Following prior self-play work, the examiner generates 1-, 2-, 3-, and 4-hop questions with a default ratio of 4:3:2:1. In each iteration (i.e., iter1, iter2, and iter3), we first train the examiner for 50 steps, then use it to generate QA data from the corresponding prompts, and subsequently train the student on the synthesized data for another 50 steps. Meanwhile, the teacher is soft-updated at every student training step. Consistent with prior self-play settings, we run only three iterations in total, yielding 150 total training steps

for each model, which is substantially fewer than baselines such as Search-R1. Throughout training, we adopt a decayed λ schedule to gradually weaken teacher guidance, allowing the student to progressively move toward regions with higher EM reward. For Qwen3-4B and Qwen3-8B, we set λ to 0.03, 0.003, and 0.002 across the three iterations, respectively. For Qwen3-4B-Instruct, we set λ to 0.1, 0.03, and 0.03. The format reward r^f for the examiner is set in the same way as the proposer’s format reward in Dr.Zero (Yue et al., 2026). Full hyperparameter details are reported in Table 6, Table 7, and Table 8.

Table 6: **Examiner hyperparameter settings.**

Steps	50
Optimizer	AdamW
Optimizer Momentum	$\beta_1, \beta_2 = 0.9, 0.999$
Warmup Ratio	0.03
Weight Decay	0.01
Learning Rate	5e-7, 1e-6
Max Gradient Norm	0.1, 1.0
Group size	1
Reward size	5
KL-Div	0
Total Train Batch Size	256
LR Scheduler	Constant with Warmup
Precision (WA)	BF16-mixed
Max Turn in Rollout	5
Max Sequence Length	4096

Table 7: **Student hyperparameter settings.**

Steps	50
Optimizer	AdamW
Optimizer Momentum	$\beta_1, \beta_2 = 0.9, 0.999$
Warmup Ratio	0.03
Weight Decay	0.01
Learning Rate	1e-6
Max Gradient Norm	0.1, 1.0
Group size in GRPO	5
KL-Div in GRPO	0, 0.001
ϵ in GRPO	0.2
Total Train Batch Size	256
LR Scheduler	Constant with Warmup
Precision (WA)	BF16-mixed
Max Turn in Rollout	5
Max Sequence Length	8192

Table 8: **Teacher hyperparameter settings.**

Precision (WA)	BF16-mixed
Soft Update Weight τ	0.05
Max Sequence Length	8192

C Further Analysis

C.1 Ablation Study

For the distillation coefficient λ in Eq. 7, we adopt a decaying schedule, as described in Appendix B. Accordingly, we conduct an ablation study on Qwen3-4B-Instruct to verify whether this decaying strategy is superior to using a fixed λ across three iterations. As shown in Table 9, compared with a constant λ , the decaying schedule not only maintains a higher evolution speed in the early stages, but also converges to better final performance in the later stages of training.

Moreover, we perform an ablation study on the top- K parameter in Eq. 12. The study is conducted on Qwen3-4B-Instruct, with K selected from 25, 50, 100, 200. As shown in Table 10, the results reveal that larger values of K do not necessarily yield better performance. Instead, the optimal performance is achieved when K is around 50.

C.2 Training Cost

Table 11 reports the average per-step training time of different methods. Across various search agents, we observe that although our approach introduces an additional teacher model compared with Dr.Zero, it does not significantly increase the per-step training time. While the teacher model performs an extra forward pass along the student’s rollout to compute token probabilities, π -Play introduces almost no additional training time compared with Dr.Zero. We attribute this to the fact that Dr.Zero lacks fine-grained feedback to penalize ineffective tool-use behaviors, causing the student model to generate redundant search queries (Fig. 4), thereby increasing training time. This effect occurs both during the computation of difficulty rewards in the examiner and during the student model’s rollout.

The reported training times were measured on a node equipped with 8 NVIDIA H20 GPUs, and we follow the same training configurations described in Appendix B.

Table 9: **Ablation study of λ .** **Bold** value indicates the top-performing result, while underline value denotes the second-best.

λ	General QA			Multi-Hop QA				Total
	NQ	TriviaQA	PopQA	HotpotQA	2WikiMQA	MuSiQue	Bamboogle	
	Iter 1							
0.100	40.1	<u>61.8</u>	42.5	<u>35.5</u>	30.1	<u>12.8</u>	<u>40.8</u>	<u>263.6</u>
0.050	39.8	62.1	42.2	34.9	31.4	11.8	41.6	263.8
0.030	40.1	61.1	<u>42.7</u>	33.7	30.4	9.85	40.0	257.9
0.010	38.5	61.6	42.5	35.4	26.6	12.4	35.2	252.2
0.005	40.1	61.1	42.0	34.9	<u>31.1</u>	10.6	32.0	251.8
0.000	40.8	61.3	42.3	34.3	27.9	11.1	31.2	248.9
Decay	<u>40.7</u>	62.1	42.9	36.7	30.1	13.2	39.2	264.9
Iter 2								
0.100	40.1	61.5	42.5	36.0	30.5	<u>13.8</u>	40.0	264.4
0.050	40.6	62.2	43.6	36.2	<u>31.3</u>	12.8	35.2	261.9
0.030	40.9	61.7	43.2	<u>37.4</u>	28.9	13.6	<u>38.4</u>	264.1
0.010	<u>41.0</u>	62.3	<u>43.7</u>	37.2	30.4	12.8	36.8	<u>264.2</u>
0.005	41.2	62.6	44.0	37.3	28.1	13.0	36.8	263.0
0.000	41.2	61.6	43.4	36.3	26.1	<u>13.8</u>	36.0	258.4
Decay	41.2	<u>62.5</u>	43.5	37.8	32.5	14.2	40.0	271.7
Iter 3								
0.100	40.8	61.7	42.3	35.6	31.0	13.4	<u>40.0</u>	264.8
0.050	41.0	61.3	42.8	35.7	30.5	<u>14.0</u>	36.0	261.3
0.030	42.5	62.0	<u>44.3</u>	38.0	29.8	14.2	37.6	<u>268.4</u>
0.010	42.5	62.1	44.5	36.5	<u>32.0</u>	12.0	36.0	265.6
0.005	<u>42.4</u>	<u>62.4</u>	43.9	37.6	28.6	13.8	38.4	267.1
0.000	41.2	62.3	43.6	37.7	29.0	13.3	38.4	265.5
Decay	41.8	63.2	44.0	38.5	32.3	13.4	44.0	277.2

C.3 Case Study

We present QCP examples generated by the examiner during the π -Play training process. The QCPs for different hop settings are presented in Fig. 5, Fig. 6, and Fig. 7. All cases are obtained from the model trained on Qwen3-4B-Instruct. In particular, we further compare the responses of the student model generated by the baseline Dr.Zero and our π -Play method on the same question, as shown in Fig. 4. Each response contains multi-turn interactions between the model and the external search engine.

Table 10: Ablation study of K . Bold value indicates the top-performing result

Top K	General QA			Multi-Hop QA				Total
	NQ	TriviaQA	PopQA	HotpotQA	2WikiMQA	MuSiQue	Bamboogle	
	Iter 1							
200	37.8	61.7	42.1	37.1	29.0	12.9	35.2	255.8
100	39.4	61.8	42.4	36.8	28.4	12.6	38.4	259.8
50	40.7	62.1	42.9	36.7	30.1	13.2	39.2	264.9
25	40.0	61.2	41.0	35.1	29.8	11.5	36.8	255.4
Iter 2								
200	41.4	61.7	43.2	35.5	29.8	11.4	36.0	259.0
100	41.9	62.2	44.4	36.8	28.5	14.0	36.8	264.6
50	41.2	62.5	43.5	37.8	32.5	14.2	40.0	271.7
25	40.5	61.9	43.0	36.9	29.9	12.9	43.2	268.3
Iter 3								
200	42.3	62.1	44.6	37.3	30.3	13.2	39.2	269.0
100	41.9	62.4	44.8	37.6	31.2	13.8	36.8	268.5
50	41.8	63.2	44.0	38.5	32.3	13.4	44.0	277.2
25	43.1	62.7	42.9	36.7	32.1	12.8	36.8	267.1

Table 11: **The training time.** We analyze the per-step training time (in seconds) for each iteration under Qwen3-4B-Instruct. Although the teacher model requires an additional forward pass along the student’s rollout to compute token probabilities, π -Play introduces only a small per-step training overhead of 4.3% relative to Dr.Zero.

	Iter1		Iter2		Iter3		Average	
	Dr.Zero	π -Play	Dr.Zero	π -Play	Dr.Zero	π -Play	Dr.Zero	π -Play
Examiner	311.6	311.6	407.8	526.3	623.4	529.9	447.6	455.9
Student (Rollout)	88.9	81.1	146.0	159.9	176.8	149.8	137.2	130.3
Teacher (Forward)	-	21.8	-	31.8	-	30.6	-	28.1
Student (Updating)	94.0	94.1	116.2	123.0	123.2	117.8	111.1	111.6
Total Time	494.5	508.6	670.0	841.0	923.4	828.1	695.9	725.9

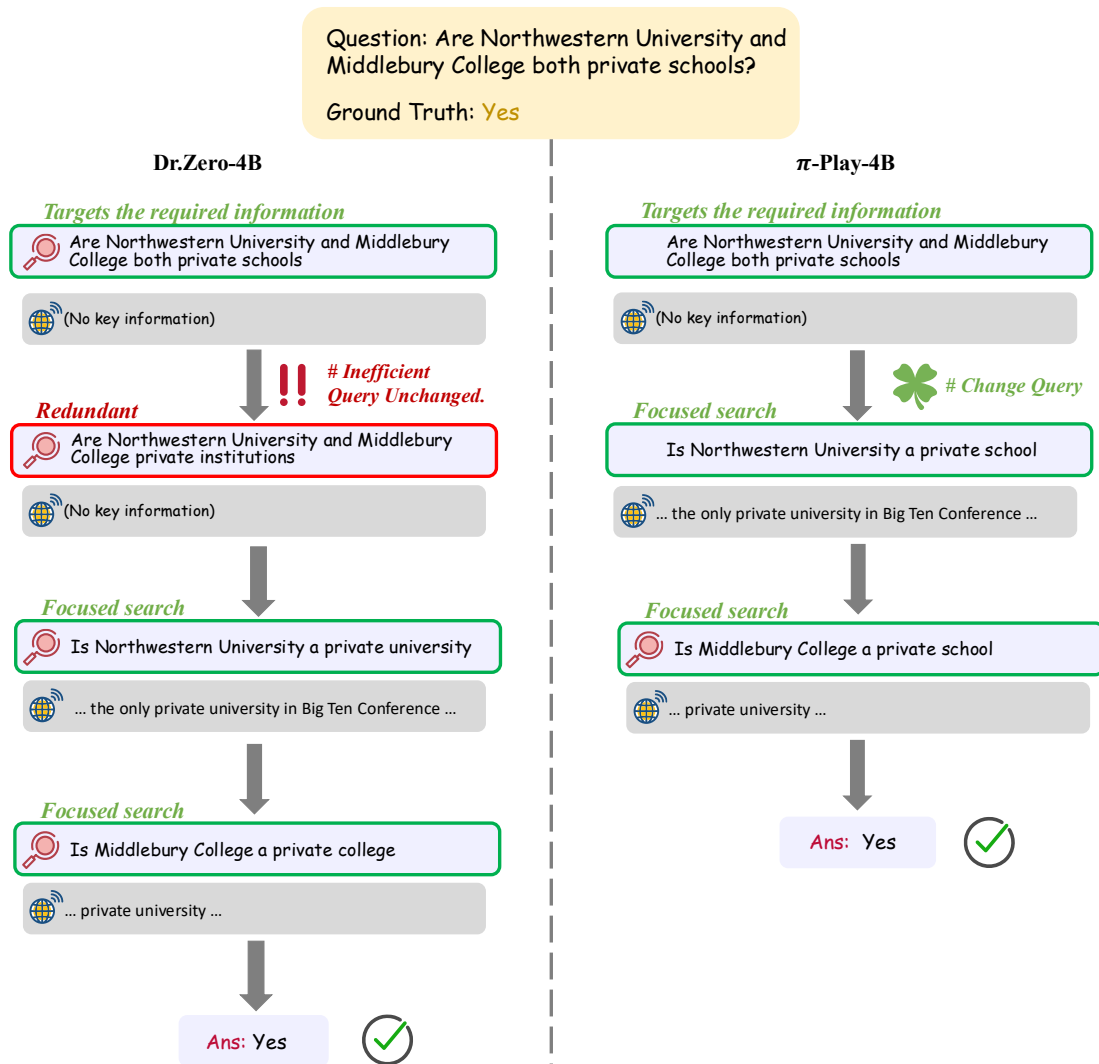


Figure 4: Side-by-side trajectories of Dr.Zero (left) and π -Play (right) on the same question. Each trajectory shows multi-turn interactions with the search engine (actions, responses, and final answer). Although both Dr.Zero and π -Play answer the query correctly, our method (right) uses fewer queries and reaches a logically structured answer with minimal redundancy.

Random Document

Now, generate a question and its answer with $n = 1$ hops starting from the following source document: (Title: "FC Barcelona Rugby League")\nwidely speculated, but purely a hypothesis, that this branch of the successful FC Barcelona club will become members of the Rugby Football League and possibly Super League should the venture prove a success in Catalonia, as they were now playing more competitive rugby, on a more regular basis.
Their first ever game was against CR Sant Cugat on 13 May 2010, a game which ended in a disappointing 30-20 loss. They earned their first win of the competition with a 44-30 win at home to GEiEG to give them a hope of qualifying for the finals. And, despite GEiEG winning ...

Examiner: Generate Task

```
<question>Which club played its first ever rugby game against CR Sant Cugat on 13 May 2010?</question>
<answer>FC Barcelona Rugby League</answer>
```

Figure 5: QCP example with hop = 1 provided by the examiner.

Random Document

Now, generate a question and its answer with $n = 2$ hops starting from the following source document: (Title: "Luna 19") Luna 19 (a.k.a. Lunik 19) (E-8-LS series), was an unmanned space mission of the Luna program. "Luna 19" extended the systematic study of lunar gravitational fields and location of mascons (mass concentrations). It also studied the lunar radiation environment, the gamma-active lunar surface, and the solar wind. Photographic coverage via a television system was also obtained. "Luna 19" was the first of the "advanced" lunar orbiters whose design was based upon the same Ye-8-class bus used for the lunar rovers and the sample collectors. For these orbiters, designated Ye-8LS, the basic "lander stage" was topped off by a ...

Examiner: Query 1

```
<tool_call>{"name": "search", "arguments": {"query_list": ["Luna 19 mission spacecraft class design"]}}</tool_call>
```

Tool Response

(Title: "Luna 19") ... Luna 19 was the first of the "advanced" lunar orbiters whose design was based upon the same Ye-8-class bus used for the lunar rovers and the sample collectors ...

Examiner: Query 2

```
<tool_call>{"name": "search", "arguments": {"query_list": ["Ye-8LS spacecraft used for lunar rovers and sample collectors"]}}</tool_call>
```

Tool Response

(Title: "Luna 21") ... Luna 21 (Ye-8 series) was an unmanned space mission, and its spacecraft, of the Luna program, also called Lunik 21, in 1973. The spacecraft landed on the Moon and deployed the second Soviet lunar rover, "Lunokhod 2". ...

Examiner: Generate Task

```
<question>Which country operated the lunar rover Lunokhod 2, which was deployed by the Ye-8 series spacecraft used in the Luna 19 mission</question>
<answer>Soviet Union</answer>
```

Figure 6: QCP example with hop = 2 provided by the examiner

Random Document

Now, generate a question and its answer with $n = 3$ hops starting from the following source document: (Title: "Chris Charsley")\nseason, including the test match against Darwen which won them promotion to the First Division. He also played for Aston Villa as a guest in 1886. Charsley had a brief spell with West Bromwich Albion, whom he joined in August 1891. He made his only appearance for Albion in a 2-1 defeat to Preston North End in November of the same year, before re-joining Small Heath the following month. On 25 February 1893, he became the first Birmingham City player-and the only player who had played for the club in its Small Heath Alliance days-to represent ...

Examiner: Query 1

```
<tool_call>{"name": "search", "arguments": {"query_list": ["Chris Charsley Birmingham City first player Small Heath Alliance"]}}</tool_call>
```

Tool Response

(Title: "History of Birmingham City F.C. (1875-1965)") ... Goalkeeper Chris Charsley, a policeman who played as an amateur and went on to serve as chief constable of Coventry, became the first Small Heath player to be capped by England. ...

Examiner: Query 2

```
<tool_call>{"name": "search", "arguments": {"query_list": ["Chris Charsley England international debut date"]}}</tool_call>
```

Tool Response

(Title: "1892-93 Small Heath F.C. season") ... Chris Charsley became the first Small Heath player to play for England, and only the second, after Caesar Jenkyns for Wales, to represent his country at full international level, when he took the field against Ireland on 27 February at Aston Villa's ground in Perry Barr, Birmingham. ...

Examiner: Query 3

```
<tool_call>{"name": "search", "arguments": {"query_list": ["Chris Charsley England international match opponent 27 February 1893"]}}</tool_call>
```

Tool Response

(Title: "1892-93 in English football") ... Chris Charsley of Small Heath, who later went on to be Chief Constable of Coventry, made his solitary England appearance in goal. ...

Examiner: Generate Task

```
<question>Which team did Chris Charsley play against in his only England international match ?</question>
<answer>Ireland</answer>
```

Figure 7: QCP example with hop = 3 provided by the examiner.

D Distillation Loss

To save GPU memory, we adopt top- K distillation following (Hübottter et al., 2026), where the top- K set is defined with respect to the student distribution:

$$\begin{aligned}
\mathbb{D}_{\text{Distill}}(\pi_\theta^S \parallel \pi_\psi^T) &= \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \text{KL}(\pi_\theta^S(\cdot \mid q, y_{i,<t}) \parallel \text{stopgrad}(\pi_\psi^T(\cdot \mid q, c, y_{i,<t}))) \\
&= \frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \sum_{\hat{y}_{i,t} \in \pi_\theta^S} \pi_\theta^S(\hat{y}_{i,t} \mid q, y_{i,<t}) \cdot \log \frac{\pi_\theta^S(\hat{y}_{i,t} \mid q, y_{i,<t})}{\text{stopgrad}(\pi_\psi^T(\hat{y}_{i,t} \mid q, c, y_{i,<t}))} \\
&= \left(\frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \sum_{\hat{y}_{i,t} \in \text{top}_K(\pi_\theta^S)} \pi_\theta^S(\hat{y}_{i,t} \mid q, y_{i,<t}) \cdot \log \frac{\pi_\theta^S(\hat{y}_{i,t} \mid q, y_{i,<t})}{\text{stopgrad}(\pi_\psi^T(\hat{y}_{i,t} \mid q, c, y_{i,<t}))} \right. \\
&\quad \left. + \underbrace{\frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \sum_{\hat{y}_{i,t} \in \pi_\theta^S \setminus \text{top}_K(\pi_\theta^S)} \pi_\theta^S(\hat{y}_{i,t} \mid q, y_{i,<t}) \cdot \log \frac{\pi_\theta^S(\hat{y}_{i,t} \mid q, y_{i,<t})}{\text{stopgrad}(\pi_\psi^T(\hat{y}_{i,t} \mid q, c, y_{i,<t}))}}_{\text{tail}} \right) \\
&\approx \left(\frac{1}{|y_i|} \sum_{t=1}^{|y_i|} \sum_{\hat{y}_{i,t} \in \text{top}_K(\pi_\theta^S)} \pi_\theta^S(\hat{y}_{i,t} \mid q, y_{i,<t}) \cdot \log \frac{\pi_\theta^S(\hat{y}_{i,t} \mid q, y_{i,<t})}{\text{stopgrad}(\pi_\psi^T(\hat{y}_{i,t} \mid q, c, y_{i,<t}))} \right. \\
&\quad \left. + \underbrace{\left(1 - \sum_{\hat{y}_{i,t} \in \text{top}_K(\pi_\theta^S)} \pi_\theta^S(\hat{y}_{i,t} \mid q, y_{i,<t}) \right) \cdot \log \frac{1 - \sum_{\hat{y}_{i,t} \in \text{top}_K(\pi_\theta^S)} \pi_\theta^S(\hat{y}_{i,t} \mid q, y_{i,<t})}{\text{stopgrad}\left(1 - \sum_{\hat{y}_{i,t} \in \text{top}_K(\pi_\theta^S)} \pi_\psi^T(\hat{y}_{i,t} \mid q, c, y_{i,<t})\right)}}_{\text{tail}} \right)
\end{aligned} \tag{12}$$

Rather than computing the full KL divergence over the entire vocabulary, we split the distillation loss into two components: the exact contribution from the student’s top- K tokens and a tail term corresponding to all remaining tokens. The tail is further approximated by collapsing the non-top- K probability mass into a single residual term. This strategy avoids storing two full copies of vocabulary logits: one for the student and one for the teacher, thereby greatly reducing memory usage. Empirically, this approximation has negligible impact on performance, since most tokens of the vocabulary are not informative at a given time (Hübottter et al., 2026). Further analysis of the choice of K can be found in Appendix C.

E Prompts

We provide the system prompts for all models in Section E.1, and the user prompts for the examiner, teacher, and student models in Sections E.2, E.3, and E.4, respectively.

E.1 System Prompts

System Prompt for the Examiner, Teacher and Student

You are a helpful assistant.

Tools

You may call one or more functions to assist with the user query.

You are provided with function signatures within <tools></tools> XML tags:

```
<tools>
{"type": "function", "function": {"name": "search", "description": "Searches the web for relevant information based on the given query.", "parameters": {"type": "object", "properties": {"query_list": {"type": "array", "description": "A list of fully-formed semantic queries. The tool will return search results for each query.", "enum": null}}, "required": ["query_list"]}, "strict": false}}
</tools>
```

For each function call, return a json object with function name and arguments within <tool_call></tool_call> XML tags:

```
<tool_call>
{"name": <function-name>, "arguments": <args-json-object>}
</tool_call>
```

Figure 8: System prompt for the examiner, teacher and student in π -play. They use the same system prompt.

E.2 Examiner Prompts

User Prompt for the Examiner

You are an expert in question generation. Craft one challenging, deterministic question and its single, unambiguous answer based on the provided source document. The logical path must start from the document and require exactly n hops (i.e., $n-1$ searches) to reach the final answer.

Definitions

1. Hop: A node in the reasoning chain. Hop 1 is the starting entity found in the document. Hop n is the final answer.

Inputs

1. n : the exact number of hops in the reasoning chain (requiring $n-1$ searches).
2. Source document: the full source text.

Process & Tools

1. Analyze the Document and Select the Starting Point
 - Read and analyze the source document.
 - Select a specific entity, event or detail explicitly mentioned in the text. This entity becomes Hop 1 (the initial clue).
2. Design the Chain Forwards
 - From Hop 1 to Hop 2: Identify a factual attribute or relation of Hop 1 that is NOT in the text but can be found via search. The result is Hop 2.
 - Iterate: Continue connecting the current Hop i to the next Hop $i+1$ using deterministic, verifiable relation found via search.
 - Stop at Hop n : Continue this process until you have exactly n hops. Hop n must be a single, canonical final answer.
3. Reasoning & Search Protocol
 - Always reason inside ` ... ` when you plan connections or receive new information.
 - For each hop transition that requires external information, issue search query using ` ... `.
 - Search results will be provided between ` ... ` by the system.
4. Output Format
 - Emit a numbered sequence of EXACTLY $n-1$ search steps. For each search i (1 to $n-1$), produce:
`<think> Reasoning step i: Identify Hop i in document/search results, formulate query to reach Hop i+1 </think>`
`<tool\_call> Query to search Hop i+1 </tool\_call>`
`[Wait for search results in <tool\_response> from system]`
 - After completing all searches and arriving at Hop n , output the question and final answer:
`<think> Final reasoning step: Confirm the chain is complete with Hop  $n$  and formulate the question </think>`
`<question> A challenging question that provides Hop 1 (the initial clue) and asks for the final answer (Hop  $n$ ) </question>`
`<answer> The single, concise final answer (Hop  $n$ ) </answer>`

Examples

1. Example template for Hop $n = 1$, i.e. no search:
`<think> [Explain how Hop 1 is selected from the source document and how the question is formulated] </think>`
`<question> [Question based solely on the text entity Hop 1] </question>`
`<answer> [Answer (Hop 1)] </answer>`
2. Example template for Hop $n = 3$, i.e. 2 searches:
`<think> [Reasoning step 1: Find Hop 1 in the source document, formulate the query to reach Hop 2] </think>`
`<tool\_call> [Search query to find Hop 2 based on Hop 1] </tool\_call>`
`[Wait for search results in <tool\_response> from system]`
`<think> [Reasoning step 2: Reason on search results to identify Hop 2 and write the next query to find Hop 3] </think>`
`<tool\_call> [Search query to find Hop 3 based on Hop 2] </tool\_call>`
`[Wait for search results in <tool\_response> from system]`
`<think> [Final reasoning step: Confirm Hop 3 in search results and formulate the question starting from Hop 1] </think>`
`<question> [Question starting with Hop 1, requiring the solver to find Hop 2 to eventually reach the Answer (Hop 3)] </question>`
`<answer> [Answer (Hop 3)] </answer>`

Critical Rules

1. Start in Document: Hop 1 must be explicitly present in the source text. Every subsequent hop must be supported by the corresponding search results.
2. Search is mandatory for $n > 1$: Each link between hops beyond Hop 1 must use the search engine.
3. Exact search count: Emit exactly $(n-1)$ `` entries, no more, no fewer.
4. No spoilers: The question must mention only Hop 1; do not include or hint at intermediate hops.
5. Clarity: The question is self-contained; the answer is concise and direct (no extra commentary, formatting or explanation).
6. Chain integrity: Each hop must depend strictly on the previous hop. No hop should be skippable or derivable without its immediate predecessor.

Now, generate a question and its answer with $n = \{\text{hops}\}$ hops starting from the following source document: {document}

Figure 9: **Initial instructions for the examiner in π -play.** Our prompt for examiner is developed based on Dr.Zero (Yue et al., 2026)

E.3 Teacher Prompts

User Prompt for the Teacher (Qwen3-4B-Instruct)

You are a helpful assistant. You will be given privileged information about the reverse solution process of the question (i.e., construction process of the question). Please pretend not to know the source document used to construct the question and the ground-truth in the reverse solution process, and answer the question again with reference to the construction process of the question.

General Rules

1. Precise Searching: When generating search queries, ensure they are specific, semantically complete, and directly target the key information you are missing.
2. Context Retention: Remember prior conversations and search results, maintaining logical consistency across multiple rounds of searching.
3. Termination Judgment: When information are sufficient to determine the answer, immediately stop searching and output the answer.
4. Reference to Privileged Information: When outputting search actions and answers, refer to the question construction process in privileged information, but do not directly output the ground-truth in the first turn or directly use extra information from the source document.
5. Search Boundary Constraint: When searching, do not introduce information that is not contained in the question or search results, even if it exists in the source document.

Example (Only showcase the logical style, please do not directly imitate the specific content)

User

This is the reverse solution process of the question (i.e., the process of generating the question) given by teacher:

=====

Now, generate a question and its answer with n = 3 hops starting from the following source document: (Title: "Steven Febey")

selected in the national draft. He fell just short of Robert Flower's club record of 272 games and as of 2007 only four people have played more games for the Demons After the 2002 season, Febey went to Bali with his teammates and was caught up in the bombings which devastated the island. Standing beside former Demons player Steven Armstrong and David Robbins, Febey was about to enter the Sari Club when the car bomb exploded...[omitted due to page size]

Reasoning step 1: Identify Hop 1 in the document, formulate the query to reach Hop 2

```
<tool_call>\n{"name": "search", "arguments": {"query_list": ["other players who played in both 1988 and 2000 AFL grand finals for Melbourne Demons"]}}\n</tool_call><|im_end|>
.....
```

Hop 2 in the document and search results is not explicitly mentioned, so we need to further refine the search. The document mentions David Neitz, the captain of Melbourne, and players like Shane Woewodin and Greg Healy. However, the search results point us to the 1988 and 2000 AFL Grand Finals, indicating that we need to look for a player specifically from these finals who also played for Melbourne. The final hop, or Hop 3, must be a well-known player from the Melbourne Demons who played in both 1988 and 2000 grand finals, which can be Robert Harvey based on historical knowledge.

Reasoning step 2: Identify Hop 2 in the search results and reformulate to find Hop 3

```
<tool_call>\n{"name": "search", "arguments": {"query_list": ["Robert Harvey Melbourne Demons 1988 2000 grand finals"]}}\n</tool_call><|im_end|>
.....
```

The search results confirm that Robert Harvey played for the Melbourne Demons in both 1988 and 2000, making him the well-known player who fits the criteria. Therefore, Hop 3 is "Robert Harvey". The question is formulated to start with the explicit entity in the document, "Steven Febey", and lead to the single, unambiguous answer, "Robert Harvey".

```
<question> Who is the only other active Melbourne Demons player to have played in both the 1988 and 2000 AFL Grand Finals , alongside Steven Febey? </question>
```

```
<answer> Robert Harvey </answer>
```

=====

Now, answer this question again by yourself, pretending you do not know the ground-truth answer, the source document. You need to start answering this question from scratch. Most importantly, pretending you do not know the ground-truth answer, the source document, and do not mention the source document when thinking. If you find you lack some knowledge, you can call a search engine by `<tool_call> query </tool_call>` and it will return the top searched results between `<tool_response>` and `</tool_response>`. You can search as many times as your want. If you find no further external knowledge needed, you can directly provide the answer inside `<answer>` and `</answer>`, without detailed illustrations. For example, `<answer> Beijing </answer>`. Question: Who wrote a number of articles, profiles, and interviews of musicians between 2004 and 2006?

Assistant (turn 1)

```
<tool_call>\n{"name": "search", "arguments": {"query_list": ["1988 AFL Grand Finals players"]}}\n</tool_call>
```

```
<tool_response> ..... </tool_response>
```

Assistant (turn 2)

```
<tool_call>\n{"name": "search", "arguments": {"query_list": ["2000 AFL Grand Finals players"]}}\n</tool_call>
```

```
<tool_response> ..... </tool_response>
```

Assistant (turn 3)

I find that Robert Harvey and Steven Febey are the active Melbourne Demons players who have played in both the 1988 and 2000 AFL Grand Finals.

```
<answer> Robert Harvey </answer>
```

Notice

Most importantly, pretending you do not know the ground-truth answer, the extra information in the source document, and do not mention the source document when thinking.

###Begin

This is the reverse solution process of the question (i.e., the process of generating the question) given by teacher:

```
=====
{demonstration}
=====
```

Now, answer this question again by yourself, pretending you do not know the ground-truth answer, the source document. You need to start answering this question from scratch. Most importantly, pretending you do not know the ground-truth answer, the source document, and do not mention the source document when thinking. If you find you lack some knowledge, you can call a search engine by `<tool_call> query </tool_call>` and it will return the top searched results between `<tool_response>` and `</tool_response>`. You can search as many times as your want. If you find no further external knowledge needed, you can directly provide the answer inside `<answer>` and `</answer>`, without detailed illustrations. For example, `<answer> Beijing </answer>`. Question: {question}

User Prompt for the Teacher (Qwen3-4B / Qwen3-8B)

You are a helpful assistant. You will be given privileged information about the reverse solution process of the question (i.e., construction process of the question). Please pretend not to know the source document used to construct the question and the ground-truth in the reverse solution process, and answer the question again with reference to the construction process of the question.

General Rules

Precise Searching: When generating search queries, ensure they are specific, semantically complete, and directly target the key information you are missing.

Context Retention: Remember prior conversations and search results, maintaining logical consistency across multiple rounds of searching.

Termination Judgment: When information are sufficient to determine the answer, immediately stop searching and output the answer.

Reference to Privileged Information: When outputting search actions and answers, refer to the question construction process in privileged information, but do not directly output the ground-truth in the first turn or directly use extra information from the source document.

Search Boundary Constraint: When searching, do not introduce information that is not contained in the question or search results, even if it exists in the source document.

Example (Only showcase the logical style, please do not directly imitate the specific content)

User

This is the reverse solution process of the question (i.e., the process of generating the question) given by teacher:

```
=====
Now, generate a question and its answer with n = 3 hops starting from the following source document: (Title: "Steven Febey")
selected in the national draft. He fell just short of Robert Flower's club record of 272 games and as of 2007 only four people have played more games for the Demons After the 2002 season, Febey went to Bali with his teammates and was caught up in the bombings which devastated the island. Standing beside former Demons player Steven Armstrong and David Robbins, Febey was about to enter the Sari Club when the car bomb exploded...[omitted due to page size]
```

Reasoning step 1: Identify Hop 1 in the document, formulate the query to reach Hop 2

```
<tool_call>\n{"name": "search", "arguments": {"query_list": ["other players who played in both 1988 and 2000 AFL grand finals for Melbourne Demons"]}}\n</tool_call><|im_end|>
.....
```

Hop 2 in the document and search results is not explicitly mentioned, so we need to further refine the search. The document mentions David Neitz, the captain of Melbourne, and players like Shane Woewodin and Greg Healy. However, the search results point us to the 1988 and 2000 AFL Grand Finals, indicating that we need to look for a player specifically from these finals who also played for Melbourne. The final hop, or Hop 3, must be a well-known player from the Melbourne Demons who played in both 1988 and 2000 grand finals, which can be Robert Harvey based on historical knowledge.

Reasoning step 2: Identify Hop 2 in the search results and reformulate to find Hop 3

```
<tool_call>\n{"name": "search", "arguments": {"query_list": ["Robert Harvey Melbourne Demons 1988 2000 grand finals"]}}\n</tool_call><|im_end|>
.....
```

The search results confirm that Robert Harvey played for the Melbourne Demons in both 1988 and 2000, making him the well-known player who fits the criteria. Therefore, Hop 3 is "Robert Harvey". The question is formulated to start with the explicit entity in the document, "Steven Febey", and lead to the single, unambiguous answer, "Robert Harvey".

```
<question> Who is the only other active Melbourne Demons player to have played in both the 1988 and 2000 AFL Grand Finals , alongside Steven Febey? </question>
```

```
<answer> Robert Harvey </answer>
=====
```

Now, answer this question again by yourself, pretending you do not know the ground-truth answer, the source document.

You need to start answering this question from scratch.

Most importantly, pretending you do not know the ground-truth answer, the source document, and do not mention the source document when thinking.

You must conduct reasoning inside `<think>` and `</think>` first every time you get new information.

After reasoning, If you find you lack some knowledge, you can call a search engine by `<tool_call> query </tool_call>` and it will return the top searched results between `<tool_response>` and `</tool_response>`.

You can search as many times as your want.

If you find no further external knowledge needed, you can directly provide the answer inside `<answer>` and `</answer>`, without detailed illustrations.

Question: Who wrote a number of articles, profiles, and interviews of musicians between 2004 and 2006?

Assistant (turn 1)

```
<think>\nTo answer this question, I need to identify the players who played for the Melbourne Demons in both 1988 and 2000 AFL Grand Finals. The question mentions Steven Febey, and we need to find another player who played in both finals. First, I will look for information about the 1988 and 2000 AFL Grand Finals and the players who participated in them. /
```

```

I will search for "1988 and 2000 AFL Grand Finals players" to find the relevant information.\n</think>

<tool_call>\n{"name": "search", "arguments": {"query_list": ["1988 AFL Grand Finals players"]}}\n</tool_call>

<tool_response> ..... </tool_response>

#### Assistant (turn 2)
<think>\nI find the information about the 1988 AFL Grand Finals players. Next, I will search for "1988 and 2000 AFL Grand
Finals players" to find the relevant information.\n</think>

<tool_call>\n{"name": "search", "arguments": {"query_list": ["2000 AFL Grand Finals players"]}}\n</tool_call>

<tool_response> ..... </tool_response>

#### Assistant (turn 3)
<think>\n \n</think>
I find that Robert Harvey and Steven Febey are the active Melbourne Demons players who have played in both the 1988 and
2000 AFL Grand Finals.
<answer> Robert Harvey </answer>

### Notice
Most importantly, pretending you do not know the ground-truth answer, the extra information in the source document, and
do not mention the source document when thinking.

###Begin
This is the reverse solution process of the question (i.e., the process of generating the question) given by teacher:

=====
{demonstration}
=====

Now, answer this question again by yourself, pretending you do not know the ground-truth answer, the source document. You
need to start answering this question from scratch. Most importantly, pretending you do not know the ground-truth answer
, the source document, and do not mention the source document when thinking. You must conduct reasoning inside <think>
and </think> first every time you get new information. After reasoning, if you find you lack some knowledge, you can call
a search engine by <tool_call> query </tool_call> and it will return the top searched results between <tool_response>
and </tool_response>. You can search as many times as your want. If you find no further external knowledge needed, you
can directly provide the answer inside <answer> and </answer>, without detailed illustrations. For example, <answer>
Beijing </answer>. Question: {question}

```

Figure 10: Initial instructions for the teacher in π -play

E.4 Student Prompts

User Prompt for the Student (Qwen3-4B-Instruct)

Answer the given question. If you find you lack some knowledge, you can call a search engine by <tool_call> query </tool_call> and it will return the top searched results between <tool_response> and </tool_response>. You can search as many times as your want. If you find no further external knowledge needed, you can directly provide the answer inside <answer> and </answer>, without detailed illustrations. For example, <answer> Beijing </answer>. Question: {question}

User Prompt for the Student (Qwen3-4B / Qwen3-8B)

Answer the given question. You must conduct reasoning inside <think> and </think> first every time you get new information. After reasoning, if you find you lack some knowledge, you can call a search engine by <tool_call> query </tool_call> and it will return the top searched results between <tool_response> and </tool_response>. You can search as many times as your want. If you find no further external knowledge needed, you can directly provide the answer inside <answer> and </answer>, without detailed illustrations. For example, <answer> Beijing </answer>. Question: {question}

Figure 11: Initial instructions for the student in π -play